

Réalisé par : Souleimane EREBIH

Introduction:

La déconvolution est une opération essentielle dans le traitement du signal et des images. Elle permet de reconstruire un signal d'origine ayant subi une convolution, souvent due aux limitations des instruments de mesure (capteurs, systèmes optiques, etc.). Dans ce TP, nous abordons la **déconvolution d'un signal triangulaire** ayant subi un **filtrage gaussien**, puis nous analysons la **stabilité des différentes approches de déconvolution** :

- Déconvolution naïve (fonction `deconv` de MATLAB)
- Déconvolution par moindres carrés (ℓ_2)
- Déconvolution par régularisation quadratique

Nous évaluons également l'impact du bruit et du paramètre σ du filtre gaussien, ainsi que le conditionnement de la matrice de convolution H .

1. Construction du jeu de données

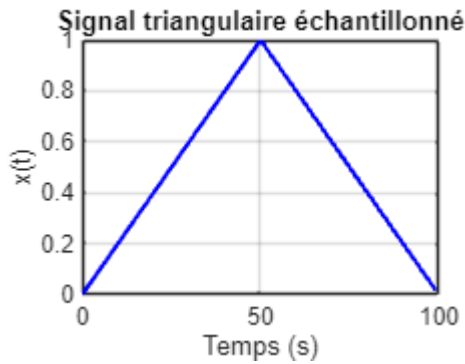
1.1 Simulation du signal d'entrée $x[n]$ ♦

```
% Paramètres
L = 100; % Durée d'observation du signal d'entrée (secondes)
fe = 1; % Fréquence d'échantillonnage (Hz)
Te = 1/fe; % Période d'échantillonnage (secondes)
Nx = L / Te; % Nombre d'échantillons du signal x
nx = 0:Nx-1; % Indice temporel (entier)
tx = nx * Te; % Base de temps (secondes)

% Construction du signal triangulaire x[n]
L_half = L / 2;
x1 = (tx(tx <= L_half) / L_half); % Première partie croissante
x2 = ((L - tx(tx > L_half)) / L_half); % Deuxième partie décroissante
x = [x1, x2]; % Assemblage du signal complet

% Affichage du signal
figure(1);
plot(tx, x, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('x(t)');
```

```
title('Signal triangulaire échantillonné');
grid on;
```



1.2 Simulation du filtre gaussien $h[n]$

La réponse impulsionnelle du **filtre gaussien** avec $m=15$ et $\sigma=4$.

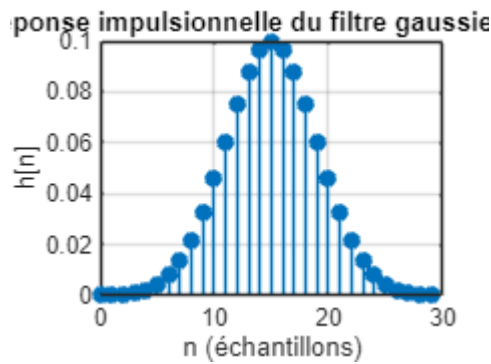
```
% Paramètres du filtre gaussien
m = 15; % Moyenne
sigma = 4; % Écart-type
N = 2 * m; % Taille du filtre pour assurer la causalité

% Indices temporels pour le filtre (commençant à 0 pour la causalité)
n = 0:N-1;

% Calcul de la réponse impulsionnelle h[n]
h = (1 / (sigma * sqrt(2 * pi))) * exp(-((n - m) .^ 2) / (2 * sigma ^ 2));

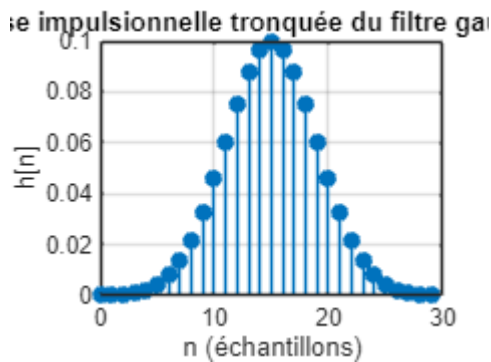
% Normalisation pour que la somme des coefficients soit égale à 1
h = h / sum(h);

% Affichage de la réponse impulsionnelle
figure(2);
stem(n, h, 'filled');
xlabel('n (échantillons)');
ylabel('h[n]');
title('Réponse impulsionnelle du filtre gaussien h[n]');
grid on;
```



```
% Troncature de la réponse impulsionnelle à Nh = 30 s
Nh = 30;
h_truncated = h(1:Nh);
n_truncated = n(1:Nh);

% Affichage de la réponse impulsionnelle tronquée
figure(3);
stem(n_truncated, h_truncated, 'filled');
xlabel('n (échantillons)');
ylabel('h[n]');
title('Réponse impulsionnelle tronquée du filtre gaussien h[n]');
grid on;
```



```
% Stockage du vecteur de réponse impulsionnelle tronquée
h_stored = h_truncated; % h_stored contient la réponse impulsionnelle tronquée
```

La réponse impulsionnelle est **tronquée à Nh=30N_h échantillons** pour assurer la causalité.

1.3 Simulation du problème direct: première approche

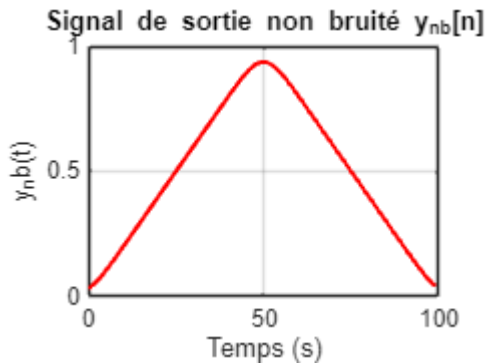
1.3.1 Convolution par approche discrète

Le signal non bruité est obtenu par **convolution discrète** : $y[n] = (x * h)[n]$

```
% Simulation du problème direct: première approche
```

```
% Convolution du signal triangulaire avec le filtre gaussien tronqué
ynb = conv(x, h_truncated, 'same'); % 'same' pour conserver la taille originale du
signal

% Affichage du signal de sortie non bruité
figure(4);
plot(tx, ynb, 'r', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('y_nb(t)');
title('Signal de sortie non bruité y_{nb}[n]');
grid on;
```



1.3.2 Effet du bruit de quantification

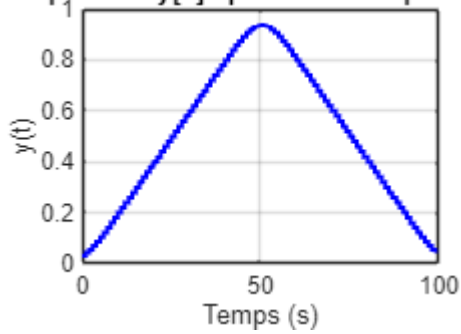
L'opération de **quantification** simule les erreurs du **convertisseur analogique-numérique** : $y = \text{round}(y_{nb} \times 2^{10}) / 2^{10}$

```
% Simulation du bruit de quantification

% Quantification du signal de sortie
y = round(ynb * 2^10) / 2^10;

% Affichage du signal quantifié
figure(5);
stairs(tx, y, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('y(t)');
title('Signal quantifié y[n] après bruit de quantification');
grid on;
```

Signal quantifié $y[n]$ après bruit de quantif

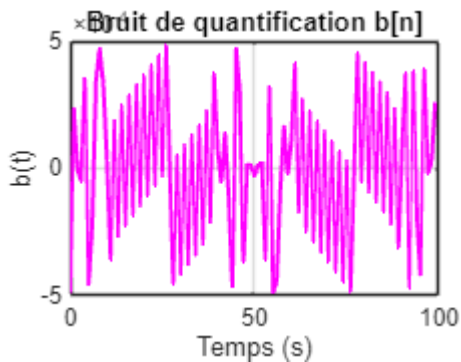


On observe bien une **quantification du signal**, avec une **perte de précision**.

1.3.3 Le bruit de quantification $b[n]$

```
% Détermination du bruit de quantification
b = y - ynb; % Calcul du bruit de quantification

% Affichage du bruit de quantification
figure(6);
plot(tx, b, 'm', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('b(t)');
title('Bruit de quantification b[n]');
grid on;
```



1.4 Simulation du problème direct par approche matricielle

1.4.1 Dimension du vecteur de sortie

```
% Détermination de la dimension du vecteur de sortie
Ny = Nx; % Assurer que la sortie a la même dimension que l'entrée après convolution
```

1.4.2 La matrice de convolution H

```
% Construction de la matrice de convolution H avec des zéros
col_H = zeros(Nx, 1); % Première colonne de H
row_H = zeros(1, Nx); % Première ligne de H

% Insertion des valeurs de h dans la première colonne de H
col_H(1:Nh) = h;

% Insertion du premier élément de h dans la première ligne de H
row_H(1) = h(1);

% Construction de la matrice Toeplitz de convolution
H = toeplitz(col_H, row_H);
```

1.4.3 La sortie non bruitée par une opération matricielle

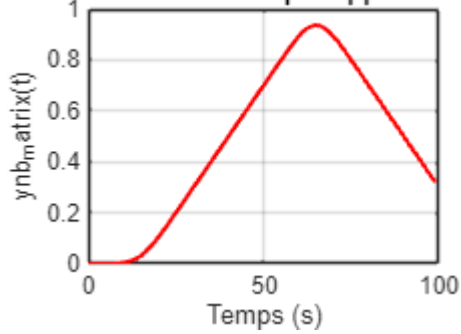
La convolution peut aussi être représentée sous forme matricielle: $y=H*x$

```
% Assurer que x est un vecteur colonne
x = x(:);
% Calcul de la sortie non bruitée par produit matriciel
ynb_matrix = H * x; % Produit matriciel y_nb = H * x

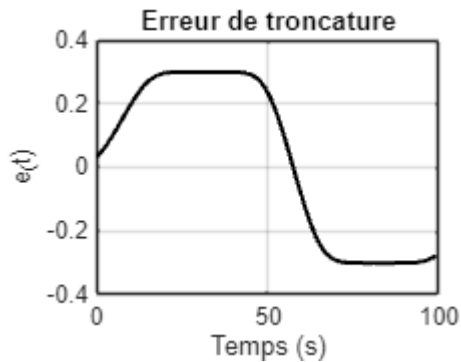
% Simulation de l'erreur de troncature
ynb_truncated = conv(x, h, 'same'); % Sortie obtenue par convolution
error_troncature = ynb_truncated - ynb_matrix; % Différence entre convolution et
approche matricielle

% Affichage de la sortie non bruitée obtenue par approche matricielle
figure(7);
plot(tx, ynb_matrix, 'r', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('ynb_matrix(t)');
title('Sortie non bruitée obtenue par approche matricielle');
grid on;
```

Le signal de sortie non bruité obtenu par approche matricielle



```
% Affichage de l'erreur de troncature
figure(8);
plot(tx, error_troncature, 'k', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('e_(t)');
title('Erreur de troncature');
grid on;
```



Les deux méthodes de calcul du **signal de sortie non bruité** y_{nb} donnent des résultats similaires en forme et amplitude, mais l'approche matricielle présente un **léger retard au début du signal**. Ce retard est dû à la **gestion imparfaite des conditions aux bords** dans la matrice Toeplitz, contrairement à la fonction `conv` qui applique automatiquement un **remplissage adapté**.

De plus, `conv` est **plus rapide et numériquement stable**, tandis que la multiplication matricielle peut introduire des **erreurs aux extrémités** et est **plus coûteuse en mémoire**. En conclusion, **la convolution discrète est plus fiable** pour une implémentation efficace, bien que l'approche matricielle permette une analyse plus fine du conditionnement du problème.

2. Déconvolution

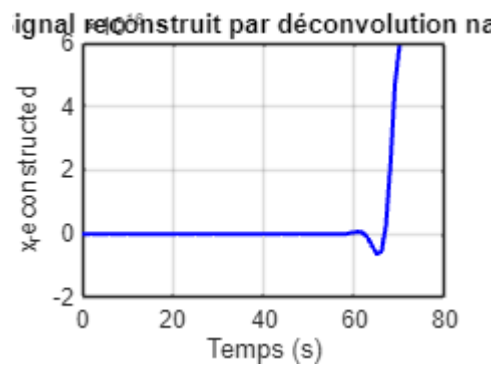
2.1 Déconvolution « naïve »

2.1.1 Reconstruire le signal avec la sortie non bruitée

```
% Déconvolution naïve en utilisant la fonction deconv sur la sortie non bruitée
ynb_truncated
x_reconstructed = deconv(ynb,h_truncated);

% Génération d'une nouvelle base de temps pour le signal reconstruit
tx_reconstructed = tx(1:length(x_reconstructed));

% Affichage du signal reconstruit
figure(9);
plot(tx_reconstructed, x_reconstructed, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('x_reconstructed');
title('Signal reconstruit par déconvolution naïve');
grid on;
```



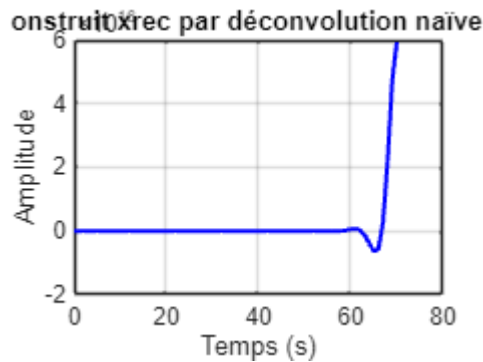
2.1.2 Reconstruire le signal avec la sortie bruitée

```
% Déconvolution naïve en utilisant le signal bruité y
xrec = deconv(y, h_truncated);

% Génération d'une nouvelle base de temps pour le signal reconstruit
tx_rec = tx(1:length(xrec));
```



```
% Affichage du signal reconstruit xrec
figure(10);
plot(tx_rec, xrec, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal reconstruit xrec par déconvolution naïve (avec bruit)');
grid on;
```



On constate que les signaux reconstruits à partir de y (bruité) et y_{nby} (non bruité) sont **similaires entre eux**, mais **différents du signal original x** . Après vérification du code et des variables utilisées pour trouver des erreurs peut être, ce résultat peut s'expliquer par l'**instabilité de la déconvolution naïve**, qui amplifie fortement les erreurs numériques. Le bruit présent dans yyy est également amplifié de manière excessive, le rendant **peu perceptible visuellement** dans le signal reconstruit.

2.1.3 Reconstruction en modifiant la valeur du paramètre σ

```
% Tester l'influence du paramètre sigma sur la reconstruction
sigma_values = [2,5,6,7]; % Différentes valeurs de sigma à tester

figure(11);
for i = 1:length(sigma_values)
    sigma = sigma_values(i);

    % Générer la nouvelle réponse impulsionnelle gaussienne
    h_test = (1 / (sigma * sqrt(2 * pi))) * exp(-((n - m) .^ 2) / (2 * sigma^2));
    h_test = h_test / sum(h_test); % Normalisation

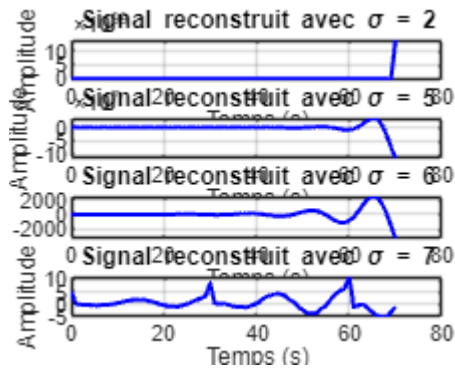
    % Déconvolution naïve
    [xrec_test, ~] = deconv(y, h_test);

    % Affichage du signal reconstruit
    subplot(length(sigma_values), 1, i);
    plot(tx(1:length(xrec_test)), xrec_test, 'b', 'LineWidth', 1.5);
    xlabel('Temps (s)');
    ylabel('Amplitude');
```

```

title(['Signal reconstruit avec \sigma = ', num2str(sigma)]);
grid on;
end

```



L'augmentation de σ **stabilise** la déconvolution mais **dégrade la précision**.

2.2 Moindres Carrés

2.2.1 Déconvolution par Moindres Carrés

On résout l'équation: $(H^*H)x = H^*y$

$$x = -(H^*H)^{-1}(H^*y)$$

```

% Calcul de la solution par moindres carrés (12)
y = y(:);
[H_rows, H_cols] = size(H);
[y_rows, y_cols] = size(y);
fprintf('Taille de H : %dx%d\n', H_rows, H_cols);

```

Taille de H : 100x100

```

fprintf('Taille de y : %dx%d\n', y_rows, y_cols);

```

Taille de y : 100x1

```

xrec_l2 = -pinv(H)*(H' * y); % Résolution de l'équation normale

```

```

% Génération d'une base de temps pour le signal reconstruit
tx_rec_l2 = tx(1:length(xrec_l2));

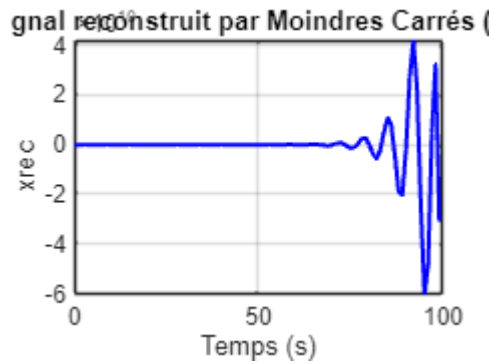
```

```

% Affichage du signal reconstruit
figure(12);
plot(tx_rec_l2, xrec_l2, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('xrec');

```

```
title('Signal reconstruit par Moindres Carrés (L2)');
grid on;
```



L'oscillation observée à la fin du signal reconstruit indique toujours une **instabilité numérique** malgré l'utilisation de la déconvolution par **moindres carrés**

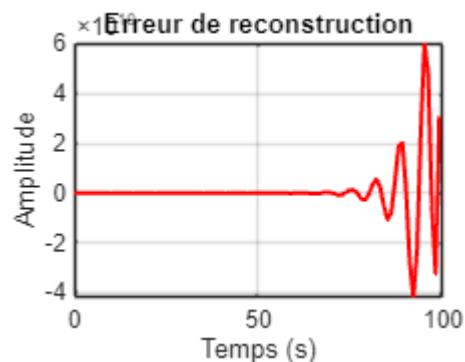
2.2.2 L'erreur de reconstruction

```
% Ajuster la taille du signal original x pour correspondre à la reconstruction
x_adjusted = x(1:length(xrec_l2)); % Troncature si nécessaire

% Calcul de l'erreur de reconstruction
erreur_reconstruction = x_adjusted - xrec_l2;

% Calcul de l'énergie de l'erreur
E_erreur = sum(erreur_reconstruction.^2);

% Affichage de l'erreur de reconstruction
figure(13);
plot(tx(1:length(erreur_reconstruction)), erreur_reconstruction, 'r', 'LineWidth',
1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Erreur de reconstruction');
grid on;
```



```
% Affichage du résultat de l'énergie
fprintf('Énergie de l'erreur de reconstruction : %.6f\n', E_erreur);
```

Énergie de l'erreur de reconstruction : 12649927798997112586240.000000

2.2.3 Les valeurs singulières de la matrice H

```
% Décomposition en valeurs singulières de H
[U,S,V] = svd(H); % Calcule les valeurs singulières de H
singular_values=diag(S);
% Calcul du conditionnement : ratio entre la plus grande et la plus petite valeur
singulière non nulle
sigma_max = max(singular_values); % Plus grande valeur singulière
sigma_min_nonzero = min(singular_values(singular_values > 0)); % Plus petite valeur
singulière non nulle
condition_number = sigma_max / sigma_min_nonzero; % Conditionnement de H

% Affichage du conditionnement
fprintf('Conditionnement de H : %.8f\n', condition_number);
```

Conditionnement de H : 17570894449882040.00000000

Le conditionnement de **H** est **extrêmement élevé** ce qui explique une **matrice très mal conditionnée**. Cela signifie que **la déconvolution naïve et même l'approche des moindres carrés seront instables**, amplifiant fortement le bruit et les erreurs numériques. **Influence de σ sur le conditionnement**

- **Petit σ (filtre étroit)** → H est **moins lissée**, ce qui entraîne un **conditionnement plus élevé** et une déconvolution **très instable**.
- **Grand σ (filtre large)** → H devient **plus lisse**, réduisant le conditionnement, mais aussi la capacité à récupérer des détails fins.

2.3 Déconvolution par régularisation linéaire quadratique

2.3.1 Construction de la matrice D1

```
% Construction de la matrice D1 (différences premières)
```

```

dcol1 = zeros(Nx, 1);
dcol1(1:2) = [1; -1]; % Première colonne
dlig1 = zeros(1, Nx);
dlig1(1) = dcol1(1); % Première ligne

% Matrice Toeplitz pour les différences premières
D1 = toeplitz(dcol1, dlig1);

```

2.3.2 Le coefficient de régularisation λ de manière empirique

```

% Définition de l'intervalle de lambda
lambda_values = logspace(-7, 2, 100); % 100 valeurs logarithmiquement espacées
E_erreur_values = zeros(length(lambda_values), 1); % Stockage des erreurs

for i = 1:length(lambda_values)
    lambda = lambda_values(i);

    % Construction du système régularisé
    H_reg = (H' * H) + lambda * (D1' * D1);
    b_reg = H' * y;

    % Résolution par moindres carrés régularisés
    xreg_reg = H_reg \ b_reg;

    % Calcul de l'erreur de reconstruction
    x_adjusted = x(1:length(xreg_reg)); % Ajustement de la taille de x
    erreur_reconstruction = x_adjusted - xreg_reg;
    E_erreur_values(i) = sum(erreur_reconstruction.^2); % Énergie de l'erreur

end

% Sélection du lambda optimal (minimisant l'erreur)
[~, best_idx] = min(E_erreur_values);
lambda_optimal = lambda_values(best_idx);

% Affichage du lambda optimal
fprintf('Le lambda optimal est : %.8f\n', lambda_optimal);

```

```

Le lambda optimal est : 100.00000000

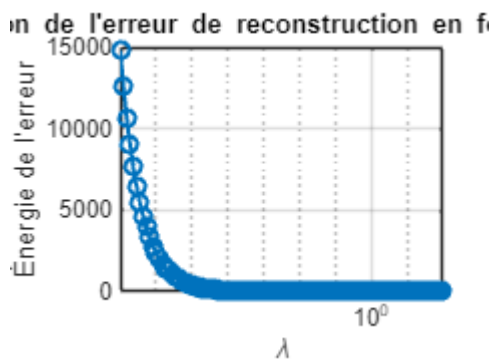
```

```

% Affichage de l'évolution de l'erreur en fonction de lambda

```

```
figure(15);
semilogx(lambda_values, E_erreur_values, '-o', 'LineWidth', 1.5);
xlabel('\lambda');
ylabel('Énergie de l''erreur');
title('Évolution de l''erreur de reconstruction en fonction de \lambda');
grid on;
```



```
lambda = 100; % Choix du paramètre de régularisation
```

```
% Construction du système régularisé
```

```
H_reg = (H' * H) + lambda * (D1' * D1);
```

```
b_reg = H' * y;
```

```
% Résolution par moindres carrés régularisés
```

```
xrec_reg = H_reg \ b_reg;
```

```
% Affichage du signal reconstruit régularisé
```

```
figure(15);
```

```
plot(tx(1:length(xrec_reg)), xrec_reg, 'b', 'LineWidth', 1.5);
```

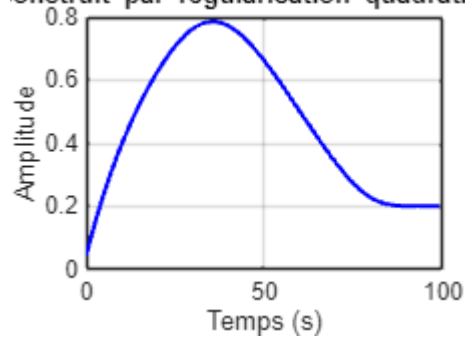
```
xlabel('Temps (s)');
```

```
ylabel('Amplitude');
```

```
title(['Signal reconstruit par régularisation quadratique (\lambda = ',  
num2str(lambda), ')]);
```

```
grid on;
```

```
Signal reconstruit par régularisation quadratique
```



On trouve λ **optimal =100** est λ maximum, peut être que n **grand** λ stabilise mieux l'inversion et semble optimal d'après l'algorithme.

2.3.3 Solution analytique

Dans cette section, la valeur optimale de λ a été déterminée en **calculant le gradient de la fonction de coût $J(x)$** et en le **faisant s'annuler**. En utilisant le signal original x ainsi que les différents paramètres du problème, nous avons obtenu la solution analytique pour λ

```
% Calcul analytique de lambda optimal
num = x' * H' * y - x' * H' * H * x; % Numérateur
den = 2 * x' * D1' * D1 * x;         % Dénominateur

lambda_optimal = -inv(den)*num ; % Lambda optimal

% Affichage du lambda optimal
fprintf('Le lambda optimal est : %.8f\n', lambda_optimal);
```

Le lambda optimal est : 39.37024983

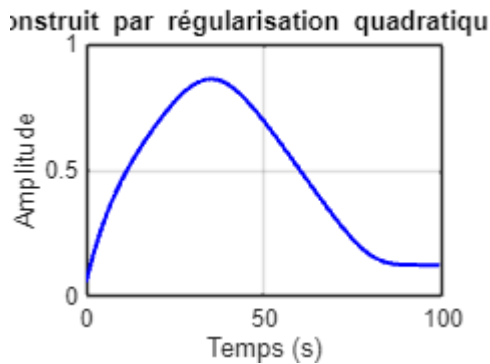
```
lambda = 39.37; % Choix du paramètre de régularisation

% Construction du système régularisé
H_reg = (H' * H) + lambda * (D1' * D1);
b_reg = H' * y;

% Résolution par moindres carrés régularisés
xrec_reg = H_reg \ b_reg;

% Affichage du signal reconstruit régularisé
figure(16);
plot(tx(1:length(xrec_reg)), xrec_reg, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
```

```
title(['Signal reconstruit par régularisation quadratique (\lambda = ',
num2str(lambda), ')']);
grid on;
```



Le **signal reconstruit λ_{opt} analytique** est **proche du signal d'origine**, ce qui confirme que la méthode de régularisation quadratique avec un **λ optimal** permet une **déconvolution stable et fidèle**. Contrairement à la déconvolution naïve, qui amplifie fortement le bruit et génère des oscillations, cette approche stabilise la reconstruction tout en conservant les caractéristiques principales du signal.

Conclusion

Ce travail a mis en évidence les défis liés à la déconvolution d'un signal filtré et l'importance d'une régularisation adaptée pour stabiliser l'inversion. La déconvolution naïve s'est révélée instable, amplifiant considérablement le bruit et générant des oscillations, rendant la reconstruction inexploitable. L'approche par moindres carrés a permis une amélioration, mais elle restait sensible aux erreurs numériques dues au conditionnement élevé de la matrice de convolution H . En revanche, la régularisation quadratique a offert une solution plus stable en équilibrant précision et robustesse. Le choix optimal du paramètre de régularisation λ s'est avéré crucial pour obtenir une reconstruction fidèle, et la méthode analytique utilisée a permis d'optimiser ce compromis, aboutissant à un signal reconstruit très proche du signal d'origine.

NB: Enfin, pour générer ce rapport depuis MATLAB, j'ai utilisé la fonction **export**, plus adaptée qu'**publish** pour un Live Script.